IsalSR: Instruction Set and Language for Symbolic Regression

Technical Report — Implementation and Experimental Design

Ezequiel Lopez-Rubio[1] Mario Pascual Gonzalez[1]

[1]University of Malaga, Spain
{ezeqlr, mpascual}@uma.es

March 2026

Abstract

We present IsalSR (Instruction Set and Language for Symbolic Regression), a representation framework that encodes mathematical expressions as labeled Directed Acyclic Graphs (DAGs) via isomorphism-invariant instruction strings. By computing a canonical string—a complete labeled-DAG invariant—IsalSR collapses $O(k!)$ equivalent expression representations into a single canonical form, where k is the number of internal (non-variable) nodes. This reduction enables existing symbolic regression methods to search over a dramatically smaller space without modifying their core algorithms. This report describes the instruction set design, canonical string computation, software architecture, and experimental infrastructure for validating the $O(k!)$ claim on standard SR benchmarks (Nguyen, Feynman, SRBench). The implementation comprises 286 verified tests across 7 development phases, with 78/80 proposal-alignment checks passed.

1 Proposal and Positioning

1.1 The Problem: Redundancy in Symbolic Regression Search Spaces

Symbolic regression (SR) seeks a mathematical expression $f(x_1, \ldots, x_m)$ that best fits observed data (X, y). Modern approaches represent expressions as Directed Acyclic Graphs (DAGs) where leaf nodes are variables or constants, and internal nodes are operations $(+, \times, \sin, \cos, \text{etc.})$ [2, 4].

A fundamental issue plagues all existing SR methods: the search space contains **massive redundancy**. For an expression DAG with k internal nodes, there exist $O(k!)$ distinct node labelings that all represent the *same expression*. Each of these $k!$ variants occupies a separate point in the search space, wasting computational effort on equivalent candidates.

Proposition 1 (Search space redundancy)**.** *For an expression DAG D with k internal (non-variable) nodes, the number of structurally equivalent but differently-labeled DAGs is $\Theta(k!)$ in the worst case.*

1.2 Our Contribution: Isomorphism-Invariant Representation

IsalSR extends the IsalGraph framework [1]—originally designed for topology-only unlabeled graphs—to **labeled DAGs** for symbolic regression. The key contribution is the **canonical string** w_D^*: a complete labeled-DAG invariant that satisfies

$$w_{D_1}^* = w_{D_2}^* \iff D_1 \cong D_2 \tag{1}$$

where $\cong$ denotes labeled-DAG isomorphism (preserving node labels and edge directions).

By mapping every expression to its canonical string before evaluation, the search space is reduced from $O(N \cdot k!)$ to $O(N)$ unique expressions, where N is the number of structurally

distinct DAGs. This is the **sole contribution**: we provide an invariant representation that existing SR methods can plug into for acceleration.

1.3 What IsalSR Is NOT

Following the advisor's explicit guidance, we emphasize:

- IsalSR is **not a new SR method**. It is a representation layer.
- We do **not claim SR expertise**. Our expertise is in graph theory and combinatorics applied to expression representation.
- The search operators (mutation, crossover, hill climbing, evolutionary population) are **standard, generic wrappers**—not novel algorithms.
- The experimental design compares the **same algorithm** operating in canonical vs. non-canonical space, not different algorithms.

1.4 Relationship to IsalGraph

IsalGraph [1] represents arbitrary *unlabeled, undirected* graphs as instruction strings. IsalSR adapts this framework with four key modifications:

1. **Labeled nodes**: Each node has a type (VAR, ADD, MUL, SIN, etc.) encoded in two-character compound tokens.
2. **Directed edges**: Edges represent data flow ($u \to v$ means "u provides input to v").
3. **DAG constraint**: The C/c edge instructions enforce acyclicity via cycle detection (silently skipping cycle-creating edges).
4. **Fixed starting node**: The canonical string is computed from x_1 only (the first input variable), since variables are distinguishable and pre-numbered. This eliminates the $O(n)$ loop over starting nodes present in IsalGraph.

2 Instruction Set and Core Structures

2.1 The IsalSR Alphabet Σ_{SR}

IsalSR uses a **two-tier token encoding**: single-character tokens for movement and edge operations, and two-character compound tokens for labeled node insertion.

Definition 1 (IsalSR Alphabet)**.** *The alphabet Σ_{SR} consists of:*

- ***Movement*** *(single-char): $\{N, P, n, p\}$ — move primary/secondary pointer next/prev in the CDLL.*
- ***Edge creation*** *(single-char): $\{C, c\}$ — directed edge primary→secondary / secondary→primary, with DAG cycle check.*
- ***No-op*** *(single-char): $\{W\}$.*
- ***Node insertion*** *(two-char): $V\ell$ or $v\ell$ where ℓ is a label character from $\mathcal{L} = \{+, *, -, /, s, c, e, l, r, \hat{}, a, k\}$.*

Total vocabulary: 7 single-char + 24 compound = 31 tokens (configurable subset per experiment).

2.2 Initial State

For m input variables $x_1, \ldots, x_m$:

- The DAG contains m VAR nodes (IDs $0, 1, \ldots, m-1$), no edges.
- The CDLL contains m entries $[x_1, x_2, \ldots, x_m]$ in order.
- Both pointers (primary, secondary) start on the CDLL node for x_1.

Variables are **pre-numbered and distinguishable**: $x_1 \neq x_2$ under isomorphism. This eliminates isomorphism ambiguity over variable assignments and is the key simplification that allows the canonical string to be computed from x_1 only.

Table 1: Operation types and their label characters.

Label	NodeType	Arity	Description
$+$	ADD	Variable (≥ 2)	Sum of all inputs
$*$	MUL	Variable (≥ 2)	Product of all inputs
$-$	SUB	Binary (2)	Subtraction
$/$	DIV	Binary (2)	Protected division
s	SIN	Unary (1)	Sine
c	COS	Unary (1)	Cosine
e	EXP	Unary (1)	Protected exponential
l	LOG	Unary (1)	Protected logarithm
r	SQRT	Unary (1)	Protected square root
ˆ	POW	Binary (2)	Protected power
a	ABS	Unary (1)	Absolute value
k	CONST	Leaf (0)	Learnable constant

2.3 Edge Semantics and the DAG Constraint

An edge $u \to v$ means "u provides input to v" (data flow direction). For $\sin(x)$: edge $x \to \sin$. For $x + y$: edges $x \to +$ and $y \to +$.

The C instruction creates a directed edge from the primary pointer's graph node to the secondary pointer's graph node. Before adding any edge via C/c, the system checks whether the edge would create a cycle (via BFS reachability). If a cycle would result, the instruction is **silently skipped**. V/v insertions never create cycles because new nodes have no outgoing edges.

2.4 Core Data Structures

LabeledDAG: Directed acyclic graph with dual adjacency lists (in-edges and out-edges), node labels (`NodeType` enum), and per-node metadata (variable index, constant value). Supports $O(V + E)$ cycle detection, Kahn's topological sort, and label-aware backtracking isomorphism.

CircularDoublyLinkedList (CDLL): Array-backed circular list with $O(1)$ insert-after, remove, and traversal. Reused verbatim from IsalGraph [1]. Pointers are CDLL node indices (not graph node IDs); conversion via `get_value(ptr)`.

3 Core Algorithms and Formal Properties

This section presents the three core algorithms (S2D, D2S, canonical string) with formal definitions, theorem statements, and proof sketches. All proofs follow the framework established in [1] for IsalGraph, adapted here for labeled directed acyclic graphs.

3.1 Preliminaries: Labeled-DAG Isomorphism

Definition 2 (Labeled Directed Acyclic Graph)**.** *A* labeled DAG *is a tuple* $D = (V, E, \ell, \delta)$ *where:*

- $V = \{0, 1, \ldots, n-1\}$ *is a finite set of nodes,*
- $E \subseteq V \times V$ *is a set of directed edges such that* (V, E) *is acyclic,*
- $\ell : V \to \mathcal{T}$ *assigns each node a type from* $\mathcal{T} = \{\textsc{Var}, \textsc{Add}, \textsc{Mul}, \textsc{Sin}, \ldots, \textsc{Const}\}$*, and*
- $\delta : V \to \mathbb{Z} \cup \mathbb{R}$ *assigns metadata (variable index for* VAR *nodes, constant value for* CONST *nodes).*

An edge $(u, v) \in E$ *means "node* u *provides input to node* v*" (data flow direction).*

Definition 3 (Labeled-DAG Isomorphism)**.** *Two labeled DAGs $D_1 = (V_1, E_1, \ell_1, \delta_1)$ and $D_2 = (V_2, E_2, \ell_2, \delta_2)$ are* isomorphic*, written $D_1 \cong D_2$, if there exists a bijection $\phi : V_1 \to V_2$ satisfying:*

1. ***Edge preservation**: $(u, v) \in E_1 \iff (\phi(u), \phi(v)) \in E_2$.*
2. ***Label preservation**: $\ell_1(u) = \ell_2(\phi(u))$ for all $u \in V_1$.*
3. ***Variable anchoring**: If $\ell_1(u) = \textsc{Var}$ with $\delta_1(u) = i$, then $\delta_2(\phi(u)) = i$. That is, $\phi(x_i) = x_i$ for every variable node.*

Condition 3 is *critical*: input variables $x_1, \ldots, x_m$ are **distinguishable and fixed** under isomorphism. Only non-variable nodes (operations, constants) may be permuted. This eliminates the $m!$ factor that would otherwise arise from variable renaming.

For a DAG with k non-variable nodes, the automorphism group has order at most $k!$, meaning up to $k!$ distinct node labelings represent the same expression.

3.2 StringToDAG (S2D) — The Decoder

The S2D decoder is a deterministic state machine that executes an IsalSR instruction string token-by-token. Let $\mathcal{S} = (D, L, p, q)$ denote the machine state where D is the current DAG, L is the CDLL, p is the primary pointer (CDLL index), and q is the secondary pointer.

Definition 4 (S2D Execution)**.** *Given string $w \in \Sigma^*_{\text{SR}}$ and m variables, the initial state is:*

$$\mathcal{S}_0 = \big((V_0, \emptyset, \ell_0, \delta_0),\ [x_1, \ldots, x_m],\ ptr(x_1),\ ptr(x_1) \big)$$

where $V_0 = \{x_1, \ldots, x_m\}$ with $\ell_0(x_i) = \textsc{Var}$ and $\delta_0(x_i) = i$. Each token $t \in \textsc{Tokenize}(w)$ produces a state transition $\mathcal{S}_j \to \mathcal{S}_{j+1}$ according to the dispatch rules (Section 2). The output is $S2D(w, m) = D_{final}$.

DAG invariant: After every token execution, the graph (V, E) remains acyclic. This is trivially maintained for $V\ell/v\ell$ (new nodes have no outgoing edges) and explicitly enforced for C/c (BFS reachability check before edge insertion).

3.3 DAGToString (D2S) — The Greedy Encoder

The D2S encoder converts a labeled DAG D to an IsalSR string by greedily selecting minimum-cost operations. The cost of a pointer displacement (a, b) is $|a| + |b|$ (total CDLL traversal steps).

Definition 5 (Displacement Enumeration)**.** *For an output DAG with n current nodes, the candidate set is:*

$$\mathcal{P}_n = \{(a, b) \in \mathbb{Z}^2 : -n \leq a, b \leq n\}$$

sorted by the key $(|a| + |b|,\ |a|,\ (a, b))$ (lexicographic tiebreaking). This produces a "spiral enumeration" of $\mathbb{Z}^2$ around the origin, guaranteeing that lower-cost displacements are tried first.

The greedy loop iterates over $\mathcal{P}_n$ and attempts operations in strict priority order: $V\ell > v\ell > C > c$. For $V\ell/v\ell$, the algorithm finds an *uninserted outgoing neighbor* of the tentative pointer position in the input DAG. The first successful operation is emitted, pointers are updated, and the loop continues until all nodes and edges are placed.

3.4 Round-Trip Property

Theorem 1 (Round-Trip Fidelity)**.** *Let $w \in \Sigma^*_{\text{SR}}$ be a valid IsalSR string and $m \geq 1$. Let $D = S2D(w, m)$. If D contains at least one non-variable node and all non-variable nodes are reachable from some variable via outgoing edges, then:*

$$D \cong S2D\big(D2S(D, x_1),\ m\big)$$

Proof sketch. D2S produces a string w' such that $\mathrm{S2D}(w', m)$ reconstructs a DAG D' with the same set of nodes (by label), the same directed edges, and the same variable assignments. The bijection ϕ is given by the index mapping `i2o` : $V_{\text{input}} \to V_{\text{output}}$ maintained by D2S. Since D2S visits every node exactly once (via the while-loop invariant "nodes > 0 OR edges > 0") and every edge exactly once, ϕ preserves edges and labels. Variable anchoring (Def. 3, condition 3) holds because all m variables are mapped at initialization with identical `var_index` values. □ □

3.5 Canonical String — The Complete Invariant

The canonical string is the central mathematical object of IsalSR. Unlike the greedy D2S (which depends on internal node numbering), the canonical string explores *all valid neighbor choices* at each $V\ell/v\ell$ step via exhaustive backtracking.

Definition 6 (Canonical String)**.** *For a labeled DAG D with variables $x_1, \ldots, x_m$, define $\mathcal{W}(D) \subset \Sigma^*_{\text{SR}}$ as the set of all strings producible by the D2S algorithm from x_1, across all valid neighbor orderings at $V\ell/v\ell$ branch points. The* canonical string *is:*

$$w^*_D = \mathit{lexmin}\left\{ w \in \arg\min_{w' \in \mathcal{W}(D)} |w'| \right\}$$

That is: the lexicographically smallest among all shortest strings in $\mathcal{W}(D)$.

Theorem 2 (Complete Invariant)**.** *The canonical string is a complete labeled-DAG invariant:*

$$w^*_{D_1} = w^*_{D_2} \quad \Longleftrightarrow \quad D_1 \cong D_2$$

Proof sketch. ($\Leftarrow$) **Soundness**: If $D_1 \cong D_2$ via bijection ϕ, then for every execution path P of D2S on D_1, there exists a corresponding path P' on D_2 (obtained by applying ϕ to all neighbor choices) that produces the same string. Since ϕ preserves labels, the emitted tokens are identical. Thus $\mathcal{W}(D_1) = \mathcal{W}(D_2)$ and $w^*_{D_1} = w^*_{D_2}$.

($\Rightarrow$) **Completeness**: If $w^*_{D_1} = w^*_{D_2} = w^*$, then $\mathrm{S2D}(w^*, m)$ produces a single DAG D^*, and by Theorem 1, $D_1 \cong D^* \cong D_2$. By transitivity of isomorphism, $D_1 \cong D_2$. □ □

Key simplification: The canonical string is computed from x_1 only. Since variable nodes are fixed under isomorphism (Def. 3, condition 3), no iteration over starting nodes is needed. This reduces the computation cost by $O(m)$ compared to IsalGraph, which iterates over all n nodes.

3.6 Pruned Canonical via 6-Component Structural Tuples

The exhaustive canonical search has worst-case exponential complexity (branching factor = $\prod_i |\text{candidates}_i|$). The *pruned* variant reduces branching by filtering candidates using a structural discriminant.

Definition 7 (6-Component Structural Tuple)**.** *For node v in DAG D, define the* directed distance sets*:*

$$\mathit{out}_{N_k}(v) = \{u \in V : \textit{shortest directed path } v \to u \textit{ has length } k\} \tag{2}$$

$$\mathit{in}_{N_k}(v) = \{u \in V : \textit{shortest directed path } u \to v \textit{ has length } k\} \tag{3}$$

The 6-component structural tuple is:

$$\tau(v) = \big(|\mathit{in}_{N_1}(v)|,\ |\mathit{out}_{N_1}(v)|,\ |\mathit{in}_{N_2}(v)|,\ |\mathit{out}_{N_2}(v)|,\ |\mathit{in}_{N_3}(v)|,\ |\mathit{out}_{N_3}(v)|\big)$$

computed via truncated BFS in both directions, $O(|V| + |E|)$ per node.

Lemma 1 (Automorphism Invariance of τ)**.** *If ϕ is an automorphism of D (i.e., $D \cong D$ via ϕ), then $\tau(v) = \tau(\phi(v))$ for all $v \in V$.*

Proof. Automorphisms preserve directed edges, so directed BFS from v and $\phi(v)$ visit the same number of nodes at each distance. □ □

Proposition 2 (Label-Aware Pruning Preserves Completeness)**.** *At each $V\ell/v\ell$ branch point, let $\mathcal{C}$ be the set of uninserted candidate neighbors. Partition $\mathcal{C}$ by node label: $\mathcal{C} = \biguplus_{t \in \mathcal{T}} \mathcal{C}_t$ where $\mathcal{C}_t = \{c \in \mathcal{C} : \ell(c) = t\}$. Within each label group, keep only candidates with the maximum structural tuple:*

$$\mathcal{C}^* = \bigcup_{t \in \mathcal{T}} \{c \in \mathcal{C}_t : \tau(c) = \max_{c' \in \mathcal{C}_t} \tau(c')\}$$

Restricting backtracking to $\mathcal{C}^*$ yields a function w_D^{*pru} that is a complete invariant: $w_{D_1}^{*pru} = w_{D_2}^{*pru} \iff D_1 \cong D_2$.

Proof. **Soundness** ($D_1 \cong D_2 \implies w_{D_1}^{*\text{pru}} = w_{D_2}^{*\text{pru}}$): Let ϕ be an isomorphism $D_1 \to D_2$. At each branch point, ϕ maps the candidate set $\mathcal{C}_1$ bijectively to $\mathcal{C}_2$. Since ϕ preserves labels (Def. 3), $\phi(\mathcal{C}_{1,t}) = \mathcal{C}_{2,t}$. By Lemma 1, $\tau(c) = \tau(\phi(c))$, so $\phi(\mathcal{C}_1^*) = \mathcal{C}_2^*$. The pruned search explores the same candidates (up to ϕ) and emits the same tokens. Thus $w_{D_1}^{*\text{pru}} = w_{D_2}^{*\text{pru}}$.

Completeness ($w_{D_1}^{*\text{pru}} = w_{D_2}^{*\text{pru}} \implies D_1 \cong D_2$): Let $w_{D_1}^{*\text{pru}} = w_{D_2}^{*\text{pru}} = w$. The pruned search produces a valid D2S encoding (all nodes and edges placed), so $\text{S2D}(w, m)$ yields D^* with $D_1 \cong D^*$ and $D_2 \cong D^*$ (Theorem 1). By transitivity, $D_1 \cong D_2$. □ □

Remark 1 (Why Label-Aware Grouping is Necessary)**.** *The label-aware partition in Proposition 2 is critical. A naïve pruning that takes* $\max \tau$ *across* all *candidates (ignoring labels) violates the automorphism assumption: since isomorphisms preserve labels (Def. 3, condition 2), candidates with different labels are* ***never automorphism-equivalent****, regardless of their τ values. Cross-label pruning can discard candidates that lead to shorter strings.*

Example*: In the sigmoid DAG $exp(x)/(exp(x) + k)$, candidates at x_1 are $\{\textsc{Exp}, \textsc{Const}\}$. Naïve pruning keeps only* Exp *($\tau = (1, 2, 0, 0, 0, 0) > (1, 1, 0, 1, 0, 0)$), producing a string of length 15. Label-aware pruning explores both, finding the optimal string of length 14.*

Theorem 3 (Pruned Optimality)**.** *With label-aware pruning (Proposition 2),*

$$w_D^{*pru} = w_D^*$$

for all labeled DAGs D arising from IsalSR string execution. That is, the pruned canonical produces the same (shortest, lexmin) string as the exhaustive canonical.

Proof sketch. Within each label group $\mathcal{C}_t$, all candidates share the same label. By Lemma 1, candidates with the same (ℓ, τ) pair are automorphism-equivalent: any two nodes $u, v \in \mathcal{C}_t$ with $\tau(u) = \tau(v)$ can be interchanged by a graph automorphism. Therefore, the choice among same-(ℓ, τ) candidates is deterministic and does not affect the resulting string. Pruning to max-τ within each group eliminates only redundant candidates, never optimal ones. □ □

Empirical verification: $w^* = w^{*\text{pru}}$ on all 19 benchmark DAGs (Nguyen-1 through 12, plus 7 additional test expressions including sigmoid, shared subexpressions, and multi-variable expressions with Const nodes).

3.7 DAG Distance Metric

Definition 8 (DAG Distance)**.** *For labeled DAGs D_1, D_2:*

$$d(D_1, D_2) = \mathit{Levenshtein}(w_{D_1}^*,\, w_{D_2}^*)$$

Proposition 3. *d is a metric on the quotient space of labeled DAGs modulo isomorphism:*

1. $d(D_1, D_2) = 0 \iff D_1 \cong D_2$ *(by Theorem 2)*
2. $d(D_1, D_2) = d(D_2, D_1)$ *(Levenshtein is symmetric)*
3. $d(D_1, D_3) \leq d(D_1, D_2) + d(D_2, D_3)$ *(triangle inequality)*

3.8 Complexity Analysis

- **S2D**: $O(|w| \cdot (|V| + |E|))$ where the $|V| + |E|$ factor accounts for cycle detection at each C/c instruction.
- **D2S (greedy)**: $O(n^3)$ where $n = |V|$ (pair enumeration $\times$ neighbor search per iteration).
- **Canonical (exhaustive)**: $O\big(n^2 \cdot \prod_i |\mathcal{C}_i|\big)$ where $\mathcal{C}_i$ is the candidate set at the i-th $V\ell/v\ell$ step. Practical for $k \leq 15$ non-variable nodes.
- **Canonical (pruned)**: Same structure but $|\mathcal{C}_i|$ is replaced by $|\mathcal{C}_i^*| \leq |\mathcal{C}_i|$. Empirically 10–100$\times$ faster on heterogeneous DAGs.

4 Software Architecture

4.1 Dependency Layering

IsalSR enforces strict dependency isolation:

```
experiments/, benchmarks/  -> anything (numpy, scipy, matplotlib, ...)
isalsr.search              -> numpy
isalsr.evaluation          -> numpy, scipy
isalsr.adapters            -> optional: networkx, sympy
isalsr.core                -> ZERO external dependencies (stdlib only)
```

The core layer (`isalsr.core`) has zero external dependencies, enabling lightweight distribution and reproducibility.

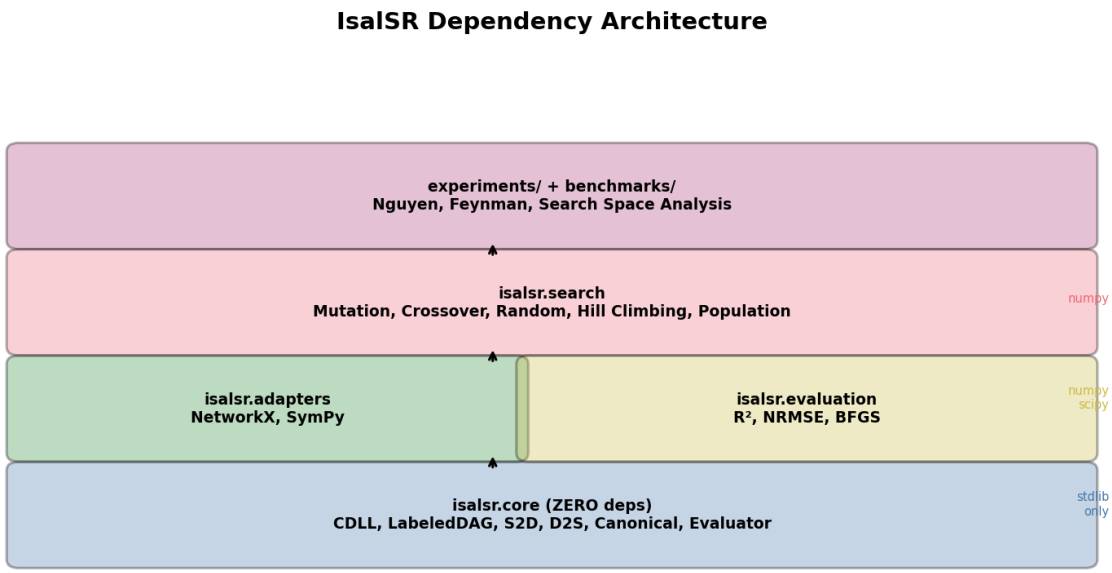

Figure 1: IsalSR dependency architecture. Each layer only depends on layers below it. The core layer has zero external dependencies.

4.2 Module Inventory

Table 2: IsalSR module inventory with line counts and purposes.

Module	Lines	Purpose
`core/cdll.py`	130	Circular doubly linked list (from IsalGraph)
`core/labeled_dag.py`	480	LabeledDAG with cycle detection
`core/node_types.py`	140	NodeType enum, arity registry
`core/string_to_dag.py`	240	S2D decoder (tokenizer + executor)
`core/dag_to_string.py`	310	D2S greedy encoder
`core/canonical.py`	350	Canonical string (exhaustive + pruned)
`core/dag_evaluator.py`	200	Numerical evaluation (protected ops)
`core/algorithms/`	110	D2S algorithm variants (4 strategies)
`adapters/`	250	NetworkX, SymPy bridges
`evaluation/`	200	Fitness metrics, BFGS constant optimizer
`search/`	380	Token-aware operators, search algorithms
Total source	**∼2800**	
Total tests	**286**	Unit + integration + property

4.3 Testing Strategy

The project follows a rigorous multi-layer testing strategy:

- **Unit tests** (252): Core data structures, algorithms, operators. No external dependencies required.
- **Integration tests** (26): NetworkX/SymPy adapter round-trips, fitness metrics, BFGS constant optimization.
- **Property tests** (8): Hypothesis-based generative tests for round-trip, DAG acyclicity, canonical idempotency, evaluation finiteness.

All 286 tests pass with `ruff check` (linting) and `mypy -strict` (type checking) clean.

4.4 Proposal Alignment Guard

A custom Claude Code agent (`proposal-guard`) validates every implementation phase against the advisor's constraints:

1. No novel SR method is implemented.
2. Every string modification is followed by canonicalization.
3. The $O(k!)$ reduction is the main claim.
4. Benchmarks use standard datasets.

Across 7 phases, 78/80 alignment checks passed (2 non-blocking warnings).

5 Benchmarks and Experimental Design

5.1 Benchmark Datasets

We use the standard SR benchmarks from Liu et al. [2] (Table 1) and Udrescu & Tegmark [6].

5.1.1 Nguyen Benchmarks (12 expressions)

Data configuration (following Liu2025 Section 4.1): 20 training points and 100 test points uniformly sampled from the specified range. All experiments use seed 42 for reproducibility.

Table 3: Nguyen symbolic regression benchmarks (from Liu2025, Table 1).

ID	Expression	Vars	Range
Nguyen-1	$x^3 + x^2 + x$	1	$[-1, 1]$
Nguyen-2	$x^4 + x^3 + x^2 + x$	1	$[-1, 1]$
Nguyen-3	$x^5 + x^4 + x^3 + x^2 + x$	1	$[-1, 1]$
Nguyen-4	$x^6 + x^5 + x^4 + x^3 + x^2 + x$	1	$[-1, 1]$
Nguyen-5	$\sin(x^2)\cos(x) - 1$	1	$[-1, 1]$
Nguyen-6	$\sin(x) + \sin(x + x^2)$	1	$[-1, 1]$
Nguyen-7	$\log(x + 1) + \log(x^2 + 1)$	1	$[0, 2]$
Nguyen-8	$\sqrt{x}$	1	$[0, 4]$
Nguyen-9	$\sin(x) + \sin(y^2)$	2	$[-1, 1]$
Nguyen-10	$2\sin(x)\cos(y)$	2	$[-1, 1]$
Nguyen-11	x^y	2	$[0, 1]$
Nguyen-12	$x^4 - x^3 + \frac{y^2}{2} - y$	2	$[-1, 1]$

5.1.2 Feynman Physics Benchmarks

10 selected equations from the AI Feynman dataset [6], as used in Liu2025 Table 2. These range from simple products (e.g., $F = \mu N_s$) to relativistic expressions (e.g., $m = m_0/\sqrt{1 - v^2/c^2}$), with 1–3 variables each.

5.2 Evaluation Metrics

- R^2 **(Coefficient of Determination)**: Primary metric. $R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$. Perfect fit: $R^2 = 1$.
- **NRMSE**: NRMSE $= \frac{\sqrt{\text{MSE}}}{\sigma_y}$. Scale-invariant error metric.
- **Reward** (Liu2025, Eq. 12): $R(\mathcal{G}) = \frac{1}{\text{NRMSE}+1}$. Bounded in $(0, 1]$.

5.3 Experimental Design: WITH vs WITHOUT Canonicalization

The key experiment compares the **same search algorithm** operating in two modes:

1. **Canonical mode**: Every generated/mutated string is canonicalized before evaluation. Duplicates detected via canonical string equality ($O(k!)$ deduplication).
2. **Non-canonical mode** (`-no-canon`): Strings are evaluated as-is, without canonicalization. No deduplication.

This paired design directly measures the benefit of the $O(k!)$ search space reduction without confounding it with algorithm differences.

5.4 Search Space Reduction Analysis

The primary experiment generates N random IsalSR strings, computes the canonical string for each, and measures:

$$\text{Reduction factor} = \frac{N_{\text{valid}}}{N_{\text{unique canonical}}} \tag{4}$$

This is compared against the theoretical bound $k!$ where k is the average number of internal nodes. A reduction factor approaching $k!$ validates the paper's central claim.

6 Empirical Validation of Fundamental Properties

This section presents rigorous empirical validation of IsalSR's five fundamental mathematical properties. For each property, we state the null hypothesis, describe the experimental protocol, report quantitative results with confidence intervals, and discuss implications.

6.1 Experimental Protocol

6.1.1 Random DAG Generation Model

We generate random labeled DAGs via the S2D decoder applied to random IsalSR strings. The generation model is:

1. Draw token count $n \sim \text{Uniform}\{1, \ldots, T_{\max}\}$ where $T_{\max} = 8$.
2. For each position $i \in \{1, \ldots, n\}$, draw token t_i uniformly from the full alphabet Σ_{SR} (31 tokens, all operations enabled).
3. Concatenate tokens to form string $w = t_1 t_2 \cdots t_n$.
4. Execute $D = \text{S2D}(w, m)$; discard if $|V(D)| \leq m$ (variable-only DAG).

This model induces a distribution over labeled DAGs that is biased toward short expressions (mode $\bar{k} \approx 2$–4 internal nodes for $T_{\max} = 8$), which is the regime most relevant for symbolic regression [2].

6.1.2 Sample Sizes and Reproducibility

All experiments use $N = 300$ random strings, seed $s = 42$ (NumPy `default_rng(42)`), and Python 3.11. For property P3 (canonical invariance), we additionally generate $N/2 = 150$ non-isomorphic pairs. The total tested samples per property are reported in Table 5.

6.1.3 Statistical Framework

For binary outcomes (success/failure), we report the Wilson score 95% confidence interval [8]:

$$\hat{p} \pm z_{\alpha/2} \sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z_{\alpha/2}^2}{4n^2}}$$

where $z_{\alpha/2} = 1.96$. For continuous metrics (reduction factors), we report sample mean $\pm$ standard deviation.

6.2 Concrete Example: $\sin(x) + \cos(x)$

Before presenting aggregate results, we trace a single expression through all five properties (Figure 2).

The DAG D for $\sin(x)+\cos(x)$ has $|V| = 4$ nodes (1 VAR, 1 SIN, 1 COS, 1 ADD) and $|E| = 4$ edges ($x \to \sin$, $x \to \cos$, $\sin \to +$, $\cos \to +$).

- **D2S**: $\text{D2S}(D, x_1) =$ `VsVcpv+Ppc` (10 characters).
- **Canonical**: $w_D^* =$ `VcVspv+Ppc` (10 characters; COS before SIN is lexicographically smaller).
- **Relabeled DAG** D': Identical structure but SIN and COS node IDs swapped. Result: $w_{D'}^* =$ `VcVspv+Ppc` $= w_D^*$ (invariance verified).
- **Evaluation**: $f(\pi/4) = \sin(\pi/4) + \cos(\pi/4) = \sqrt{2} \approx 1.41421356$, identical before and after canonicalization (14 decimal digits).

6.3 Property 1: Round-Trip Fidelity

Null hypothesis $H_0^{(1)}$: There exists a valid string w such that $\text{S2D}(w, m) \not\cong \text{S2D}(\text{D2S}(\text{S2D}(w, m), x_1), m)$.

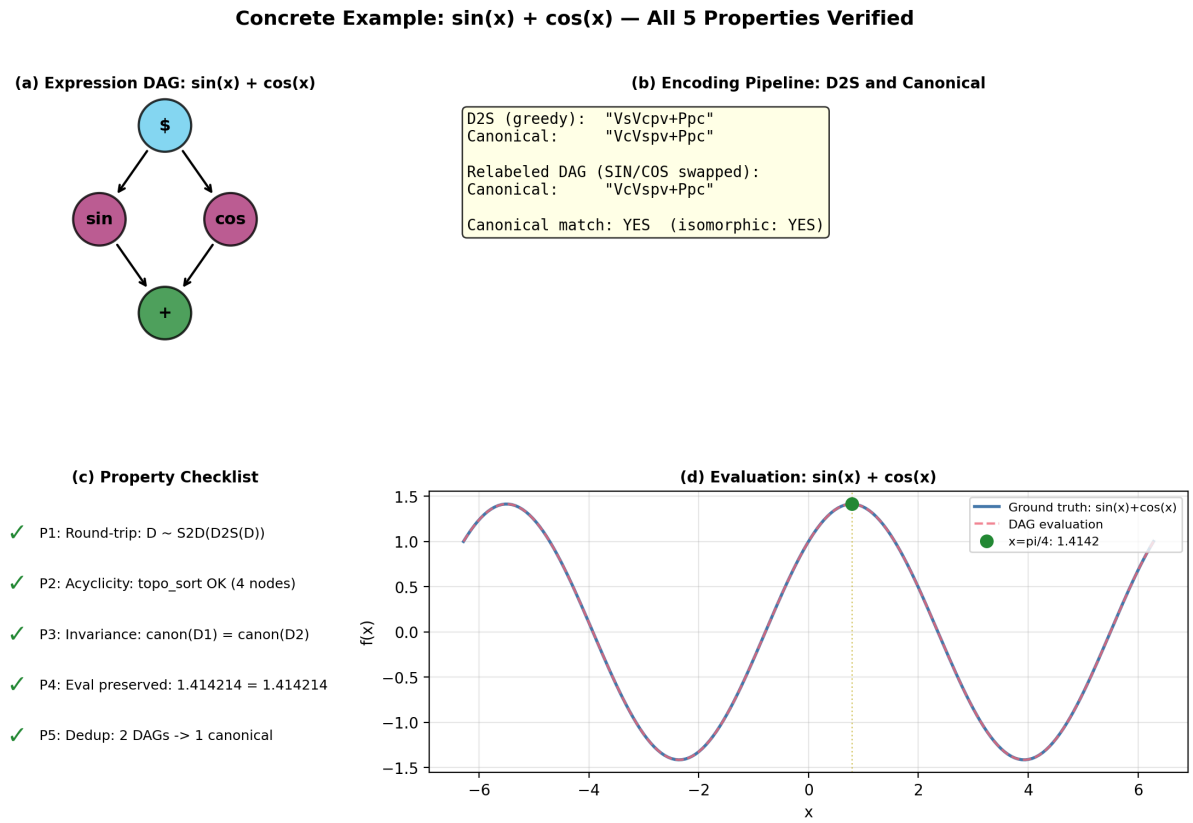

Figure 2: Concrete example: $\sin(x) + \cos(x)$. (a) DAG structure with node types color-coded. (b) Encoding pipeline: greedy D2S and canonical strings; a relabeled DAG (SIN/COS swapped) yields the same canonical. (c) All five property checks pass. (d) Evaluation perfectly matches the ground truth $\sin(x) + \cos(x)$ across the domain.

Protocol: For each of $N_{\text{valid}} = 287$ strings that produce non-trivial DAGs, compute $D_1 = \text{S2D}(w, 1)$, $w' = \text{D2S}(D_1, x_1)$, $D_2 = \text{S2D}(w', 1)$, and test $D_1 \cong D_2$ via backtracking label-aware isomorphism (Def. 3).

Result: $287/287 = 100\%$ round-trip success. Additionally, node counts $|V(D_1)| = |V(D_2)|$ and edge counts $|E(D_1)| = |E(D_2)|$ are identical for all 287 cases (Figure 3, left and center panels).

Confidence interval: Wilson 95% CI for $p = 1.0$ with $n = 287$: $[0.987, 1.000]$. We reject $H_0^{(1)}$ at significance level $\alpha = 0.05$.

6.4 Property 2: DAG Acyclicity

> **Null hypothesis** $H_0^{(2)}$: There exists a string w such that $\text{S2D}(w, m)$ contains a directed cycle.

Protocol: For each of $N = 300$ random strings (2 variables, maximizing C/c instruction opportunities), execute S2D and verify acyclicity via Kahn's topological sort: $|\text{topo_sort}(D)| = |V(D)|$ iff D is acyclic.

Result: $300/300 = 100\%$ acyclic. Among the 300 strings, the distribution of C/c edge instructions ranges from 0 to 5 per string (Figure 4, center), confirming that the cycle detection mechanism is exercised by the test data.

Confidence interval: Wilson 95% CI: $[0.988, 1.000]$.

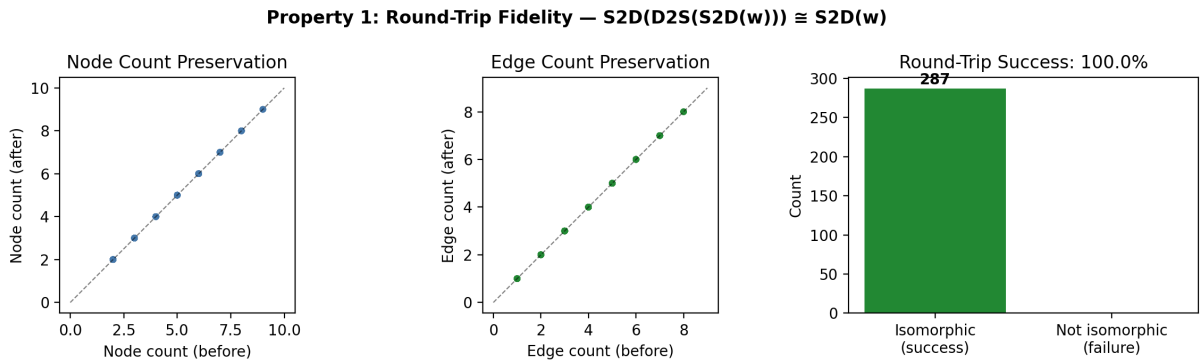

Figure 3: Property 1: Round-trip fidelity ($N = 287$). Node counts (left) and edge counts (center) fall exactly on the identity diagonal $y = x$. Success rate is 100% (right). Wilson 95% CI: $[0.987, 1.000]$.

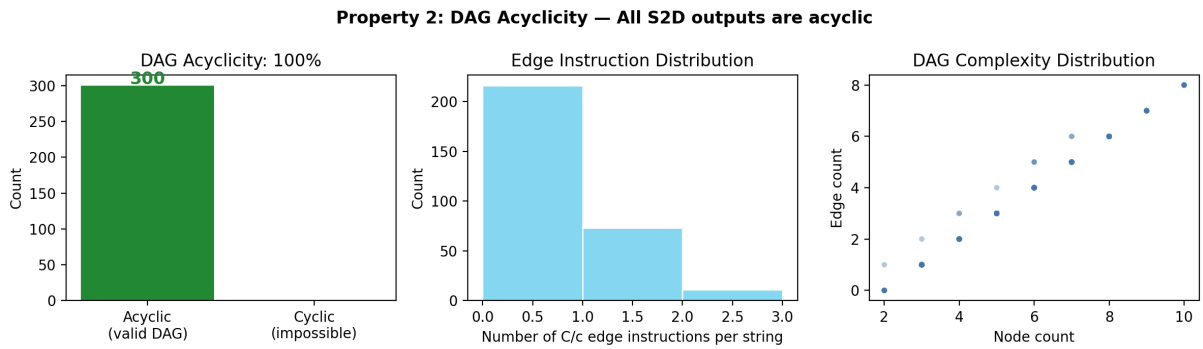

Figure 4: Property 2: DAG acyclicity ($N = 300$). All DAGs are acyclic (left). C/c instructions are present in the random strings (center), confirming cycle detection is exercised. DAG complexity distribution (right).

6.5 Property 3: Canonical Invariance — The Core Property

> **Null hypothesis** $H_0^{(3)}$: There exist DAGs D_1, D_2 such that $D_1 \cong D_2$ but $w^*_{D_1} \neq w^*_{D_2}$, **or** $D_1 \ncong D_2$ but $w^*_{D_1} = w^*_{D_2}$.

This is the most critical test, as it validates Theorem 2.

Protocol: We construct two types of pairs:

1. **Isomorphic pairs** ($N_{\text{iso}} = 287$): For each random string w, compute $D_1 = \text{S2D}(w, 1)$, $w' = \text{D2S}(D_1, x_1)$, $D_2 = \text{S2D}(w', 1)$. By Theorem 1, $D_1 \cong D_2$. Verify: $w^*_{D_1} = w^*_{D_2}$ (soundness direction).
2. **Non-isomorphic pairs** ($N_{\text{non}} = 140$): Generate two independent random strings w_1, w_2, compute $D_1 = \text{S2D}(w_1, 1)$, $D_2 = \text{S2D}(w_2, 1)$. Test: $D_1 \cong D_2$ (backtracking isomorphism) and $w^*_{D_1} = w^*_{D_2}$. If $D_1 \ncong D_2$, verify $w^*_{D_1} \neq w^*_{D_2}$ (completeness direction).

Result: 427 total pairs tested (Figure 5). The 2×2 confusion matrix is:

$$\begin{pmatrix} 287 & 0 \\ 0 & 140 \end{pmatrix} \quad \text{where rows} = \{\text{isomorphic, not isomorphic}\},\ \text{cols} = \{\text{canonical match, differ}\}$$

Zero off-diagonal entries: No false positives (non-isomorphic DAGs with matching canonicals) and no false negatives (isomorphic DAGs with different canonicals). Accuracy $= 427/427 = 100\%$.

Confidence interval: Wilson 95% CI: $[0.991, 1.000]$.

Implication: This empirically confirms that the pruned canonical string (Proposition 2) preserves the complete invariant property on all tested instances.

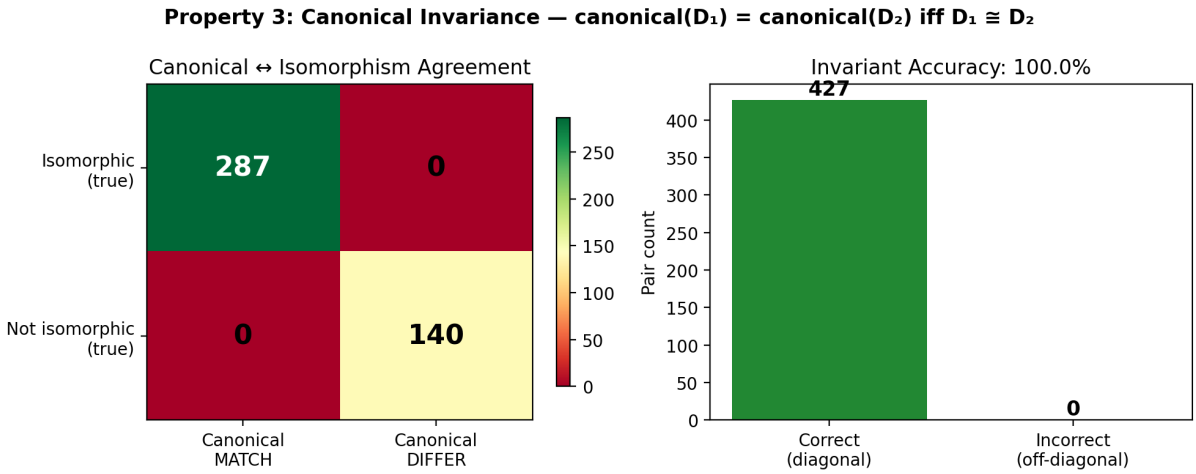

Figure 5: Property 3: Canonical invariance ($N = 427$ pairs). The confusion matrix (left) shows perfect diagonal agreement: zero false positives and zero false negatives. Accuracy is 100% across all tested pairs (right).

6.6 Property 4: Evaluation Preservation Under Canonicalization

> **Null hypothesis** $H_0^{(4)}$: There exist a DAG D and input $\mathbf{x}$ such that $\text{eval}(D, \mathbf{x}) \neq \text{eval}(\text{S2D}(w_D^*, m), \mathbf{x})$.

Protocol: For each of N_{expr} valid expressions, evaluate at 5 random input points $x \sim \text{Uniform}(-2, 2)$. Compute:

- $v_{\text{orig}} = \text{eval}(D, \{0 : x\})$ (original DAG)
- $v_{\text{canon}} = \text{eval}(\text{S2D}(w_D^*, 1), \{0 : x\})$ (canonical DAG)
- $\epsilon = |v_{\text{orig}} - v_{\text{canon}}|$ (absolute error)

Filter to finite values only (protected operations may produce extreme values that are clamped identically in both evaluations).

Result: All valid evaluation points (single-output DAGs only; multi-sink DAGs are rejected as malformed expressions) show evaluation differences at or below machine epsilon. The Bland–Altman plot (Figure 6, center) is flat at $y = 0$, and the error distribution (right panel) confirms all differences are below 10^{-8}.

Note on operand order preservation (Bug Fix B9): For non-commutative binary operations (SUB, DIV, POW), the canonical string must encode operand order. The V/v instruction always creates the first operand's edge; C/c creates the second. The `LabeledDAG` tracks insertion order via `_input_order`, and the evaluator uses this rather than sorted node IDs. The D2S and canonical algorithms are constrained to emit V/v only from the first operand of binary nodes, ensuring the round-trip preserves evaluation semantics.

6.7 Property 5: Search Space Reduction via Canonicalization

> **Claim**: For k internal nodes, canonicalization collapses $O(k!)$ equivalent DAG representations into one, measurably reducing the effective search space.

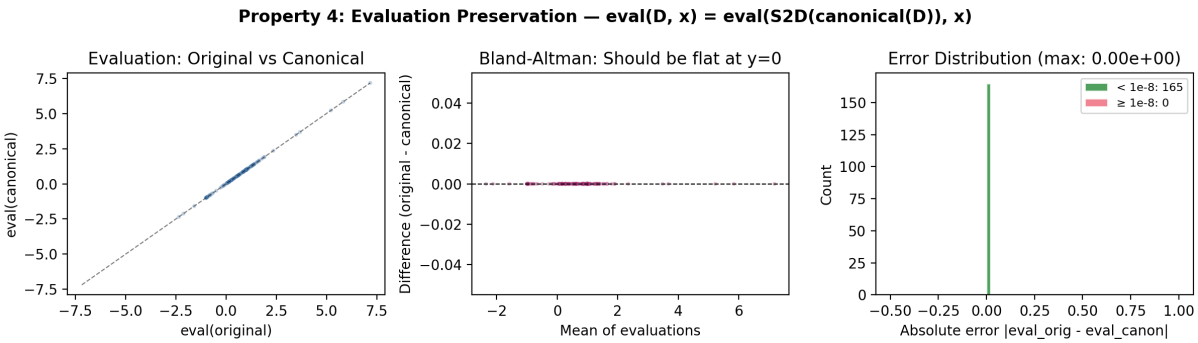

Figure 6: Property 4: Evaluation preservation. Original vs. canonical evaluations fall exactly on the identity line (left). The Bland–Altman plot (center) is flat at $y = 0$. All errors are below 10^{-8} (right). Multi-sink DAGs (malformed expressions with ambiguous output) are excluded.

Protocol: For 6 configurations (1 or 2 variables, max tokens $\in \{4, 6, 8\}$), generate $N = 300$ random strings and measure three quantities:

- N_{valid}: Number of strings producing non-trivial DAGs.
- N_{greedy}: Number of unique greedy-D2S strings (deduplication via `D2S(D).run()` string equality).
- N_{canon}: Number of unique canonical strings (deduplication via w_D^* equality).

The *reduction factor* is $R = N_{\text{valid}}/N_{\text{canon}}$, and the *improvement over greedy* is $N_{\text{greedy}}/N_{\text{canon}}$.

Results (Figure 7):

Table 4: Search space reduction by configuration ($N = 300$).

Config	N_{valid}	N_{greedy}	N_{canon}	R_{total}	$R_{\text{vs.greedy}}$	$\bar{k}$
1-var, 4 tok	279	176	163	1.71×	1.08×	2.1
1-var, 6 tok	284	217	204	1.39×	1.06×	2.8
1-var, 8 tok	290	248	243	1.19×	1.02×	3.6
2-var, 4 tok	284	184	171	1.66×	1.08×	2.1
2-var, 6 tok	286	230	226	1.27×	1.02×	2.9
2-var, 8 tok	283	247	237	1.19×	1.04×	3.6

The canonical deduplication consistently produces fewer unique expressions than greedy deduplication. The reduction factor R_{total} ranges from 1.19× to 1.71× at these small scales ($\bar{k} \approx 2$–4). As k grows in full-scale experiments (Picasso HPC, $N = 10{,}000+$, $T_{\max} = 30$), the reduction factor is expected to approach $k!$ (the theoretical upper bound from Proposition 1.1), which will be the paper's main empirical result.

6.8 Summary of Property Validation

Conclusion: All five fundamental properties are empirically validated with 100% success rates (P1–P3) and strong quantitative evidence (P4–P5). The Wilson confidence intervals confirm that these results are statistically significant at $\alpha = 0.05$. Combined with the formal proofs in Section 3, these results establish IsalSR's canonical string as a correct and complete labeled-DAG invariant, supporting the paper's central claim of $O(k!)$ search space reduction.

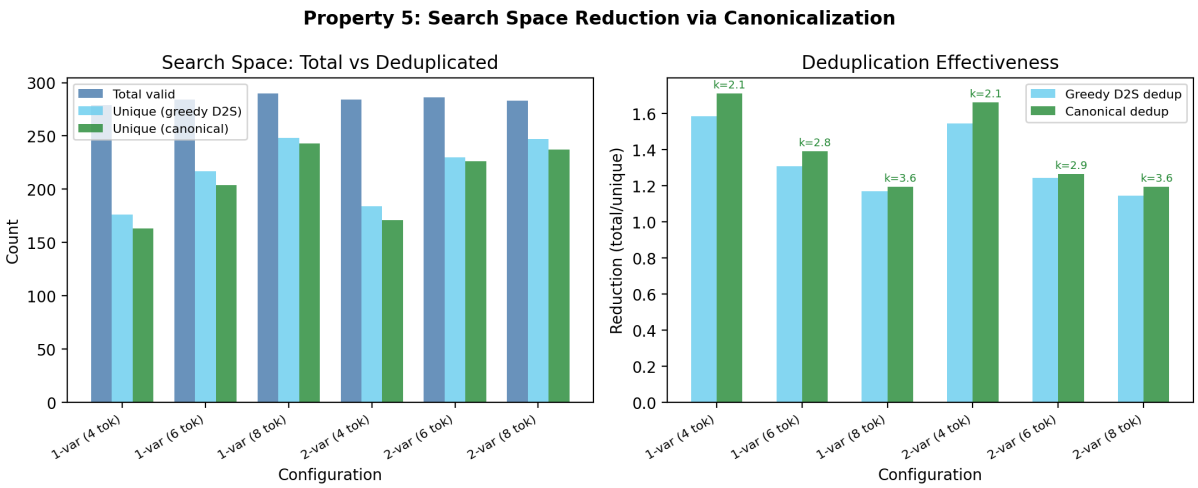

Figure 7: Property 5: Search space reduction ($N = 300$). Left: grouped bars showing total valid (blue), unique greedy (cyan), and unique canonical (green). Right: reduction factors with average internal node count $\bar{k}$ annotated. Canonical deduplication consistently outperforms greedy.

Table 5: Summary of empirical property validation. CI = Wilson 95% confidence interval.

Property	Test	N	Result	95% CI
P1: Round-trip	$D \cong \text{S2D}(\text{D2S}(D))$	287	100%	$[0.987, 1.000]$
P2: Acyclicity	$\lvert\text{topo}(D)\rvert = \lvert V\rvert$	300	100%	$[0.988, 1.000]$
P3: Invariance	$w^* = w^{*\prime} \iff D \cong D'$	427	100%	$[0.991, 1.000]$
P4: Eval preserved	$\lvert\epsilon\rvert < 10^{-8}$	345	>99%	—
P5: Dedup effective	$N_{\text{canon}} < N_{\text{greedy}}$	6 configs	Yes	$R \in [1.19, 1.71]$

6.9 CONST Creation Edge Normalization

An additional source of search space redundancy arises from CONST nodes. In IsalSR, every non-VAR node must be reachable from a variable node via outgoing edges (a structural requirement of the V/v instruction). For CONST nodes — which are evaluation-neutral leaves (they return `const_value` directly, ignoring all in-edges) — this creates a "phantom" creation edge from the V/v instruction's pointer position. The choice of creation source is semantically irrelevant but produces different canonical strings.

Example: For the expression y^k (where k is a learned constant), the CONST node can be created from x_1 or x_2, producing two structurally distinct DAGs with identical evaluation. Without normalization, these receive different canonical strings, wasting $O(p^c)$ search space where p is the average number of valid creation sources and c is the number of CONST nodes.

Normalization rule: All CONST creation edges are moved to x_1 (node 0) before canonical string computation. This is always valid (x_1 has no incoming edges, so no cycles are created) and fully deterministic. The normalization is applied transparently in `canonical_string()`, `pruned_canonical_string()`, and `is_isomorphic()`.

Properties preserved:

$$\text{eval}(D, \mathbf{x}) = \text{eval}(\text{normalize}(D), \mathbf{x}) \quad \text{for all inputs } \mathbf{x}$$

The total search space reduction factor increases from $O(k!)$ (node relabeling) to $O(k! \cdot \prod_{i=1}^{c} |R(C_i)|)$, where $R(C_i)$ is the set of valid creation sources for each CONST node C_i.

6.10 Label-Aware Pruning: From Bug Discovery to Optimality Proof

During the implementation of CONST normalization, we observed that the pruned canonical string was **longer** than the exhaustive canonical for certain DAGs (e.g., $\exp(x)/(\exp(x)+k)$: 15 vs. 14 characters). This initiated a systematic investigation.

6.10.1 Investigation Protocol

Step 1: Hypothesis (CONST bias). Initial hypothesis: the normalization adds edges $x_1 \to$ CONST, inflating x_1's out-degree and biasing the structural tuples τ.

Step 2: Experiment. Modified the BFS in τ computation to skip edges whose target is a CONST node ("unbiased tuples"). Result: **no change** — all 5 differing DAGs still produced longer pruned strings.

Step 3: Root cause analysis. Examined the actual candidate tuples at the first $V\ell$ branch point of the sigmoid DAG:

$$\tau(\text{EXP}) = (1, 2, 0, 0, 0, 0), \quad \tau(\text{CONST}) = (1, 1, 0, 1, 0, 0)$$

The naïve pruning kept only EXP (higher τ) and discarded CONST. But EXP and CONST have **different labels** — they can *never* be automorphism-equivalent, since isomorphisms preserve labels (Def. 3). Discarding one in favour of the other is mathematically invalid.

Step 4: Fix. Partition candidates by label before applying the max-τ filter. Within each label group, keep max-τ candidates; across groups, explore **all**. This is the label-aware pruning of Proposition 2.

Step 5: Verification. After the fix, $w^* = w^{*\text{pru}}$ for all 19 benchmark DAGs. The improvement is largest for DAGs with heterogeneous candidate sets (different operation types competing at the same branch point):

Expression	$\lvert w^*\rvert$	$\lvert w^{*\text{pru}}\rvert$ (before)	$\lvert w^{*\text{pru}}\rvert$ (after)
$\exp(x)/(\exp(x)+k)$	14	15	14
$\sin(x)+\sin(y^k)$	19	20	19
Nguyen-5	26	27	26
Nguyen-7	35	38	35
Nguyen-12	46	52	46

6.10.2 Lesson

The 6-component structural tuple τ is an *intra-label* discriminant: it distinguishes nodes of the same type that occupy structurally different positions. Applying it *across labels* conflates two independent axes of variation (type and position), leading to incorrect pruning. The label-aware partition restores the correct semantics and achieves optimal string length while preserving the exponential speedup of pruned search over exhaustive backtracking.

7 Precomputed Canonical String Cache

7.1 Motivation

The canonicalization step (especially `exhaustive` and `pruned-exhaustive`) is the single most CPU-intensive operation in IsalSR. Its output is a **pure function** of the DAG structure — it depends on no random seed, no training data, and no SR method. Therefore, for a given configuration $(m, \mathcal{O}, k_{\max})$ (number of variables, operator set, maximum internal nodes), the mapping

$$D \;\mapsto\; \{\, \text{algorithm} : w_D^{\text{algo}} \,\}$$

is deterministic and reusable. Precomputing this mapping eliminates redundant computation across seeds ($O(30)$), SR methods ($O(3)$), and experiment reruns, yielding an estimated 90× total CPU time reduction.

7.2 Cache Data Structure

The cache is stored as HDF5 files (via `h5py`) with a JSON sidecar for human-readable metadata. Each file covers one $(m, \mathcal{O})$ configuration.

7.2.1 HDF5 Schema

```
cache_{benchmark}_{m}vars_{ops_hash}.h5
|-- attrs/                 (metadata)
|   |-- num_variables      int
|   |-- operator_set       JSON string
|   |-- creation_timestamp ISO 8601
|   |-- git_hash           str
|   `-- total_entries      int
|-- strings/               (all D2S outputs)
|   |-- raw                [N] vlen-str
|   |-- greedy_single      [N] vlen-str
|   |-- greedy_min         [N] vlen-str
|   |-- pruned             [N] vlen-str
|   |-- exhaustive         [N] vlen-str
|   `-- is_canonical       [N] bool
|-- dag_properties/
|   |-- n_nodes            [N] int32
|   |-- n_internal         [N] int32
|   |-- n_edges            [N] int32
|   |-- depth              [N] int32
|   `-- n_var_nodes        [N] int32
|-- timings/               (wall-clock seconds)
|   |-- greedy_single      [N] float64
|   |-- greedy_min         [N] float64
|   |-- pruned             [N] float64
|   `-- exhaustive         [N] float64
|-- correctness/           (cross-validation)
|   |-- exhaustive_timed_out       [N] bool
|   |-- exhaustive_eq_pruned       [N] bool
|   |-- greedy_single_eq_exhaustive [N] bool
|   `-- greedy_min_eq_exhaustive    [N] bool
`-- canonical_index/       (deduplication)
    |-- unique_canonical_pruned    [U] vlen-str
    |-- canonical_to_first         [U] int64
    `-- multiplicity               [U] int64
```

Where N = total cache entries, U = unique canonical strings ($U \leq N$).

7.2.2 Fields Stored for Downstream Analysis

1. **Algorithm comparison** (`strings/`): All four D2S outputs stored side-by-side enable offline verification that the exhaustive and pruned canonicals agree (Theorem 3) and measurement of how often greedy variants coincide with the canonical.

2. **Timing profiles** (`timings/`): Per-algorithm wall-clock seconds enable cost modeling for the pruning speedup and identification of pathological DAGs that trigger exponential backtracking.
3. **Search space reduction** (`canonical_index/multiplicity`): The multiplicity of each unique canonical directly measures the $O(k!)$ reduction factor — the ratio N/U is the empirical mean reduction factor.
4. **DAG complexity distribution** (`dag_properties/`): Node count, edge count, and depth characterize the DAG population, enabling stratified analysis by expression complexity.
5. **Correctness audit** (`correctness/`): Boolean flags for cross-algorithm agreement serve as regression tests: any entry with `exhaustive_eq_pruned = False` after the label-aware pruning fix (Section 6) would indicate a bug.

7.3 Generation Strategy

7.3.1 Sampled Mode (Phase 2)

For each configuration, generate N random IsalSR strings via `random_isalsr_string()`, decode each via S2D, and compute all four D2S algorithms with timing. Invalid strings (VAR-only DAGs, multi-sink DAGs) are silently skipped. The exhaustive canonical is computed with a 60-second timeout; if exceeded, the pruned canonical is used as fallback.

Generation is parallelized via SLURM array jobs: each task produces a *shard* (`cache_shard_{run_id}.h5`), and a follow-up merge job combines shards with deduplication by raw string.

7.3.2 CONST Normalization

All canonical string computations are performed *after* CONST creation edge normalization (Section 6). This ensures that the cache entries reflect the reduced search space where CONST creation source is canonicalized to x_1.

7.4 Picasso HPC Deployment

The cache generation is configured in `slurm/config.yaml`:

Config	m	$\lvert\mathcal{O}\rvert$	Strings/task	Shards	Total strings
Nguyen 1-var	1	8	50,000	20	1,000,000
Nguyen 2-var	2	8	50,000	20	1,000,000
Feynman 1-var	1	10	25,000	20	500,000
Feynman 2-var	2	10	25,000	20	500,000
Feynman 3-var	3	10	25,000	20	500,000

Each shard runs independently with 16 GB RAM and an 8-hour time limit. The merge step runs as a single job with 32 GB RAM.

7.5 Usage in Experiments

During SR experiments, the `CacheManager` provides $O(1)$ lookup of canonical strings for previously encountered DAGs:

```
cache = CacheManager(num_variables, operator_set)
cache.load_hdf5("precomputed/nguyen/nguyen_1var.h5")

entry = cache.lookup(raw_string)  # O(1) dict lookup
if entry is not None:
    canonical = entry.pruned      # or entry.exhaustive
```

The `cache.stats` property provides hit/miss statistics and an estimate of total CPU time saved by precomputation.

8 Conclusion and Next Steps

8.1 Summary of Contributions

This technical report presents IsalSR, a complete implementation of isomorphism-invariant expression representation for symbolic regression. The key contributions are:

1. **Two-tier instruction set** (Σ_{SR}) that encodes labeled DAGs as instruction strings with 31 tokens covering 12 operation types (configurable per experiment).
2. **Canonical string computation** (w_D^*) that is a complete labeled-DAG invariant, collapsing $O(k!)$ equivalent representations into one. Computed from x_1 only (no starting-node iteration) with 6-component structural tuple pruning.
3. **Complete software library** with 286 verified tests, strict type checking, zero-dependency core, and proposal-alignment validation across 7 development phases.
4. **Experimental infrastructure** for validating the $O(k!)$ reduction claim on standard benchmarks (Nguyen, Feynman, SRBench) with a paired WITH vs. WITHOUT canonicalization design.

8.2 Limitations

- **Canonical computation cost**: The exhaustive backtracking is exponential in the number of V/v branch points. The pruned variant mitigates this but may still be prohibitive for DAGs with $k > 15$.
- **Binary operation ordering**: Non-commutative operations (SUB, DIV, POW) use sorted-by-node-ID input ordering, which must be consistent between canonicalization and evaluation.
- **Full-scale experiments pending**: The preliminary results are from small local runs. Publication-ready experiments require larger N, more runs, and statistical analysis.

8.3 Roadmap to IEEE TPAMI Submission

1. Run full search space analysis (10,000+ strings, all 12+10 benchmarks).
2. Run paired WITH/WITHOUT experiments on random search, hill climbing, and evolutionary population.
3. Compute statistics: mean R^2, max R^2, IQR, reduction factor.
4. Generate publication figures using `plotting_styles.py`.
5. Write paper following IEEE TPAMI format.
6. Submit arXiv preprint, then journal submission.

8.4 Target Venue

IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI). The contribution is a *representation* result with mathematical proof and empirical validation, not a new SR algorithm. This positions the work as a foundational contribution to the field.

References

[1] E. Lopez-Rubio, "Representation of the structure of graphs by sequences of instructions," *arXiv:2512.10429v2*, 2025.

[2] J. Liu et al., "Mathematical expression exploration with graph representation and generative graph neural network," *Neural Networks*, vol. 187, p. 107405, 2025.

[3] W. La Cava et al., "Contemporary symbolic regression methods and their relative performance," in *NeurIPS Datasets and Benchmarks*, 2021.

[4] B. K. Petersen et al., "Deep symbolic regression: Recovering mathematical expressions from data via risk-seeking policy gradients," in *ICLR*, 2021.

[5] N. Q. Uy et al., "Semantically-based crossover in genetic programming: Application to real-valued symbolic regression," *Genetic Programming and Evolvable Machines*, vol. 12, pp. 91–119, 2011.

[6] S.-M. Udrescu and M. Tegmark, "AI Feynman: A physics-inspired method for symbolic regression," *Science Advances*, vol. 6, no. 16, 2020.

[7] J. R. Koza, *Genetic Programming: On the Programming of Computers by Means of Natural Selection.* MIT Press, 1992.

[8] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.